# TRACKFLOW + GTMXpert

## Hoja de ruta de producto

De la captura de elemento al documento de etiquetado

---

Documento de arquitectura para Antigravity

ESES Agency · Marzo 2026

# 0. Visión general — dos productos, un flujo

**El error más habitual al construir una herramienta así:** intentar hacer todo en un solo producto. TRACKFLOW y GTMXpert hacen cosas distintas y deben poder evolucionar por separado.

| Producto | Su trabajo |
|---|---|
| TRACKFLOW | Captura. Escucha clicks, extrae HTML del elemento seleccionado y genera un payload estructurado con lo que el analista quiere trackear. |
| GTMXpert | Procesamiento. Recibe el payload de TRACKFLOW, aplica las reglas del estándar ESES y genera el tag + trigger + variable listos para GTM, más el bloque de documentación. |

**El flujo completo en una frase:** Analista hace clic sobre un elemento → TRACKFLOW captura el contexto → GTMXpert genera la implementación → el analista la pega en GTM y en el documento del cliente.

**Por qué este orden importa:** si mezclas la captura y el procesamiento en un solo paso, pierdes la capacidad de revisar qué capturaste antes de que la IA lo interprete. Ese momento de revisión es donde el analista añade criterio. Quitarlo convierte la herramienta en una caja negra.

# 1. Arquitectura del sistema

## 1.1 Los tres componentes

| Componente | Tecnología recomendada | Responsabilidad |
|---|---|---|
| Extensión Chrome (TRACKFLOW) | Chrome Extension MV3 + JS vanilla | Captura click, extrae HTML limpio, envía payload al backend |
| Backend API | Node.js / Python (FastAPI) en Railway o Render | Recibe payload, llama a Claude API, devuelve implementación estructurada |
| App web (GTMXpert) | Next.js o React + Supabase | Panel del analista: historial, editor, exportación a Word |

## 1.2 El payload — el formato que lo conecta todo

Todo el sistema pivota alrededor de un objeto JSON que TRACKFLOW genera y GTMXpert consume. Definir bien este objeto es la decisión de arquitectura más importante del proyecto.

**Payload TRACKFLOW → GTMXpert (formato propuesto):**

```
{
  // Contexto del cliente (rellena el analista en la extensión)
  "client": {
    "name": "Argenta Cerámica",
    "url":  "argentaceramica.com",
    "gtm_container": "GTM-WF6J29K6",
    "ga4_measurement_id": "G-XXXXXXX",
    "has_facebook_pixel": true
  },

  // Lo que el analista quiere trackear (describe en lenguaje natural)
  "intent": "Clic en el botón de descarga del catálogo",
  "event_name_suggestion": "file_download",

  // Lo que TRACKFLOW capturó automáticamente
  "captured": {
    "click_element_html": "<a class=\"listado-catalogos__grid__item__descargas\"
href=\"/link.php?...\" target=\"_blank\">Descargar</a>",
    "parent_html": "<div class=\"listado-catalogos__grid__item\"><h4>Catálogo General
2025</h4>...</div>",
    "click_url": "https://argentaceramica.com/link.php?link=...pdf",
    "page_path": "/descargas/catalogos/",
    "page_title": "Catálogos - Argenta Cerámica",
    "timestamp": "2026-03-05T10:23:00Z"
  }
}
```

## 1.3 El prompt del sistema — la base de conocimiento de GTMXpert

**Este es el núcleo del producto.** El prompt del sistema es el documento que le dice a Claude (o Gemini) cómo tiene que comportarse. Contiene las reglas del estándar ESES, los patrones de código correctos, los errores que nunca debe cometer y el formato exacto de salida.

**Lo que ya tenemos construido** y que se convierte directamente en prompt:

| Material existente | Cómo se usa en el prompt |
|---|---|
| Las 27 reglas del estándar (R01–R27) | Sección de restricciones: 'Nunca hagas X, siempre haz Y' |
| Los patrones de código corregidos (Argenta) | Sección de ejemplos: few-shot examples con el código que sí funciona |
| La estructura de la guía de etiquetado | Sección de formato de salida: exactamente qué debe devolver el modelo |
| Los errores identificados por paso | Sección de errores comunes: patrones que el modelo debe detectar y evitar |

 **Ventaja competitiva real:** La mayoría de herramientas de GTM con IA mandan el HTML al modelo y esperan que lo resuelva. GTMXpert manda el HTML más 27 reglas explícitas, ejemplos de código validados y el historial del cliente. La diferencia en calidad de salida es enorme.

# 2. TRACKFLOW — la extensión de Chrome

**Qué hace exactamente:** el analista activa la extensión en la página del cliente, hace clic sobre el elemento que quiere trackear, y TRACKFLOW captura todo el contexto necesario para que GTMXpert pueda generar la implementación correcta.

## 2.1 El flujo de uso paso a paso

| Paso | Qué ocurre |
|---|---|
| 1. Activar en la página | El analista hace clic en el icono de TRACKFLOW. La extensión inyecta un overlay que resalta los elementos clickables. |
| 2. Seleccionar el elemento | El analista hace clic sobre el elemento que quiere trackear (botón, formulario, enlace...). |
| 3. Rellenar contexto mínimo | Un popup pequeño pide: intent (qué quieres trackear), event_name sugerido, notas opcionales. Todo lo demás se captura solo. |
| 4. Enviar a GTMXpert | Con un clic, el payload se envía al backend. La respuesta llega en segundos. |
| 5. Revisar y aceptar | El analista ve la implementación propuesta (tag + trigger + variable + código), la revisa y la acepta o edita. |
| 6. Añadir al documento | La implementación aceptada se añade a la guía del cliente en construcción. |

## 2.2 Lo que TRACKFLOW captura automáticamente

| Campo capturado | Por qué es necesario |
|---|---|
| HTML del elemento clickado (limpio) | El selector CSS más estable se extrae de aquí. GTMXpert elige la estrategia de captura según la estructura del HTML. |
| HTML del elemento padre (2 niveles) | Permite detectar si hay texto descriptivo (como el <h4> del catálogo) que no está en el elemento clickado pero que hay que leer. |
| URL completa de la página | Para generar la condición de Page Path del trigger automáticamente. |
| Texto visible del elemento | Para sugerir el nombre del evento y el valor del parámetro principal. |
| Atributos relevantes (href, id, class, data-*) | Los data attributes son la fuente de información más estable y menos frágil. |

## 2.3 Lo que NO captura TRACKFLOW — lo que pone el analista

 **Distinción importante:** TRACKFLOW captura contexto técnico. El analista aporta contexto estratégico. La herramienta no puede saber si un clic en un botón verde es una conversión de alto valor o un simple toggle de UI. Ese juicio es humano.

| El analista decide | Ejemplo |
|---|---|
| Qué evento de GA4 asignar | ¿Es un file_download, un generate_lead o un select_content? |
| Si tiene etiqueta de Facebook y cuál | ¿Es un Lead? ¿Un ViewContent? ¿Sin Facebook porque es navegación interna? |
| Los parámetros adicionales que importan | ¿Hay que capturar el nombre del catálogo? ¿La provincia del formulario? |
| La prioridad del evento en la guía | ¿Es un evento core o un evento de engagement secundario? |

# 3. GTMXpert — el motor de procesamiento

**Qué hace exactamente:** recibe el payload de TRACKFLOW, aplica las reglas del estándar ESES Agency y devuelve una implementación completa, lista para pegar en GTM y para incluir en el documento del cliente.

## 3.1 La salida de GTMXpert — qué devuelve exactamente

Para cada elemento capturado, GTMXpert devuelve un bloque estructurado con cinco secciones:

| Sección de salida | Contenido |
|---|---|
| 1. Análisis del elemento | Qué ha detectado en el HTML, qué estrategia de captura recomienda y por qué (con justificación basada en las reglas R01–R27). |
| 2. Variable(s) GTM | Nombre, tipo y código JavaScript completo. Solo las variables necesarias, sin extras. |
| 3. Trigger | Tipo de trigger, condiciones exactas con los valores reales del HTML capturado, condición de Page Path si aplica. |
| 4. Etiqueta(s) | GA4 con todos los parámetros. Facebook con el Standard Event correcto (o justificación de por qué no). Nombre y trigger asignado. |
| 5. Bloque de documentación | El párrafo listo para insertar en el Word del cliente: rationale del evento + checklist de validación. |

## 3.2 La estructura del prompt del sistema

El prompt del sistema es el documento más importante del proyecto. Aquí está su estructura propuesta:

| Sección del prompt | Contenido y longitud estimada |
|---|---|
| ROL Y CONTEXTO | Eres GTMXpert, el asistente de etiquetado GTM de ESES Agency. Tu trabajo es analizar elementos HTML y generar implementaciones de tracking correctas siguiendo el estándar ESES. (~200 palabras) |
| REGLAS TÉCNICAS (R01–R10) | Las 10 reglas técnicas en formato instrucción directa: 'Nunca uses .includes(). Usa siempre indexOf() !== -1. Nunca uses regex. Usa indexOf() + substring()...' (~600 palabras) |
| REGLAS GTM (R11–R20) | Reglas de estructura GTM, formularios y triggers. Con ejemplos de código CORRECTO vs INCORRECTO para los patrones más críticos. (~800 palabras) |

| | |
|---|---|
| REGLAS GA4 Y FACEBOOK (R21–R27) | Reglas de eventos y Facebook Pixel. Especialmente la distinción InitiateCheckout/Lead y cuándo no usar Facebook. (~400 palabras) |
| EJEMPLOS FEW-SHOT (3–5 casos) | Casos completos: payload de entrada → implementación correcta. Tomados directamente de la guía de Argenta corregida. (~1500 palabras) |
| FORMATO DE SALIDA | Instrucción exacta de qué devolver y en qué formato JSON. El modelo NO improvisa el formato. (~300 palabras) |
| ERRORES COMUNES A EVITAR | Lista de los 10 errores más frecuentes identificados en la auditoría de Argenta. (~400 palabras) |

## 3.3 Modelo: Claude vs Gemini

 **La pregunta que mencionaste sobre Gemini:** Ambos modelos pueden hacer este trabajo. La diferencia práctica está en la integración: Claude API es más sencilla de usar vía Anthropic directamente. Gemini tiene ventaja si quieres integrarte con Google Workspace (Sheets, Drive). La recomendación es Claude para el núcleo de generación de código — el razonamiento sobre HTML y código JavaScript es donde Claude destaca más.

| Criterio | Recomendación |
|---|---|
| Generación de variables JS y triggers | Claude Sonnet — mejor razonamiento de código |
| Integración con Google Sheets / Drive | Gemini via Google AI Studio si es necesario |
| Coste por consulta | Claude Sonnet: ~0.003€ por implementación completa (estimado) |
| Velocidad de respuesta | Ambos < 5 segundos para este tipo de payload |
| Veredicto | Empezar con Claude API. Añadir Gemini solo si hay caso de uso específico de Google Workspace. |

# 4. La base de conocimiento — lo que ya tienes construido

**Buenas noticias:** el trabajo más difícil de una herramienta de este tipo no es el código. Es el conocimiento. Y ese conocimiento ya existe — está en las guías de clientes, en la auditoría de Argenta, en las 27 reglas del estándar.

**Lo que hay que hacer** es estructurarlo para que el modelo lo consuma de forma consistente.

## 4.1 Fuentes de conocimiento existentes

| Fuente | Qué aporta al sistema |
|---|---|
| Guía Argenta corregida (este documento) | Los few-shot examples más completos: 7 pasos, 24 etiquetas, código validado y probado. |
| Guía residencialperu132.es | Caso de uso inmobiliario: formularios de contacto, galerías, mapas. Amplía el rango de casos cubiertos. |
| Guía eses.agency (CF7 conversacional) | El caso más técnicamente complejo: formulario oculto, listener Custom HTML, MutationObserver. Muy valioso como ejemplo de edge case. |
| Las 27 reglas R01–R27 | El corazón del prompt del sistema. Ya están redactadas con justificación. Solo hay que reformatearlas como instrucciones directas. |
| Auditoría de Argenta (el React tracker) | Los 35 errores identificados se convierten en la sección 'Errores comunes a evitar' del prompt. |

## 4.2 Cómo se actualiza la base de conocimiento

Con cada nuevo cliente, la base de conocimiento crece. El proceso propuesto:

| Cuándo | Qué se añade |
|---|---|
| Al terminar una guía nueva | Se añade como nuevo few-shot example al prompt si cubre un patrón no visto antes (nuevo tipo de formulario, nuevo plugin de WordPress, etc.) |
| Al identificar un error nuevo | Se añade a la sección de errores comunes del prompt con código correcto vs incorrecto. |
| Al detectar un patrón recurrente | Se extrae como regla nueva (R28, R29...) con su justificación. |
| Periodicidad de actualización | Revisión del prompt cada 4-6 semanas o cuando se acumulen 3+ patrones nuevos. |

# 5. Hoja de ruta — fases de construcción

El proyecto se divide en cuatro fases. Las dos primeras son las más importantes — sin ellas el resto no funciona bien. Las dos últimas convierten la herramienta interna en un producto compartible con el equipo.

## FASE 1 · El prompt funciona [2–3 semanas]

☐ **Redactar el prompt del sistema v1:** Tomar las 27 reglas + los 3 ejemplos de Argenta y escribirlo en formato instrucción. Sin código todavía.

☐ **Testear manualmente:** Pegar payloads de ejemplo directamente en Claude.ai y validar que la salida es correcta. Iterar el prompt hasta que el 90% de casos salga bien.

✅ **Criterio de éxito:** El prompt genera una implementación correcta (sin errores de las 27 reglas) para 10 casos de prueba distintos.

## FASE 2 · El backend funciona [2–3 semanas]

☐ **API mínima:** Un endpoint POST /generate que recibe el payload JSON y devuelve la implementación. Node.js + Express o Python + FastAPI. Deploy en Railway (gratis hasta cierto límite).

☐ **Autenticación básica:** API key simple por usuario del equipo ESES. Sin OAuth complejo todavía.

☐ **Formato de salida validado:** El backend devuelve JSON estructurado con las secciones: analysis, variables, trigger, tags, documentation.

✅ **Criterio de éxito:** Hacer una llamada desde Postman con un payload real y recibir una implementación completa y correcta.

## FASE 3 · TRACKFLOW v1 [3–4 semanas]

☐ **Extensión Chrome básica:** Manifest v3. Un botón de activación. Al hacer clic sobre un elemento, captura el HTML del elemento y su padre, la URL de la página y el texto visible.

☐ **Popup mínimo:** Input de texto para el 'intent', selector de event_name (dropdown con las opciones más comunes), botón de enviar. Sin más.

☐ **Conexión al backend:** La extensión llama al endpoint /generate y muestra la respuesta en un panel lateral.

✅ **Criterio de éxito:** Un analista del equipo puede capturar un elemento real, enviarlo y recibir la implementación sin salir del navegador.

## FASE 4 · App GTMXpert completa [4–6 semanas]

☐ **Historial por cliente:** Supabase como base de datos. Cada implementación generada se guarda asociada al cliente. El analista puede ver el historial y editar.

 **Exportación a Word:** El mismo sistema que genera las guías actuales, pero alimentado por las implementaciones aprobadas. Un clic = guía completa del cliente.

 **Multi-usuario:** Login por email. Cada miembro del equipo ESES tiene su cuenta. Las implementaciones se comparten por cliente.

✅ **Criterio de éxito:** El equipo usa GTMXpert para el 100% de los clientes nuevos. Las guías se generan en menos de 1 hora.

# 6. Lo que le pasas a Antigravity

**No le pases este documento completo todavía.** Un documento de arquitectura de 30 páginas al inicio de un proyecto es una forma segura de malentendidos costosos. Lo que funciona es dividirlo en entregas pequeñas con criterios de éxito claros.

## 6.1 Entregable 1 — el contrato de datos

Lo primero que Antigravity necesita es el formato exacto del payload. Antes de tocar código, hay que acordar este formato y no cambiarlo durante las fases 1 y 2.

> □ **Qué enviarle:** El JSON del payload documentado en la sección 1.2 de este documento. Pedirle que lo revise y proponga cambios si ve algo que complica la implementación de la extensión Chrome. Acordar el formato final antes de empezar.

## 6.2 Entregable 2 — la Fase 2 (backend)

El backend es lo que Antigravity puede construir mientras tú trabajas el prompt en paralelo. Es independiente — solo necesita recibir un JSON y devolver otro JSON. No necesita saber nada de GTM para construirlo.

| Spec técnica para Antigravity | Detalle |
|---|---|
| Endpoint | POST /api/generate |
| Input | JSON con el payload definido en sección 1.2 |
| Output | JSON con: { analysis, variables[], trigger, tags[], documentation } |
| Auth | Header: X-API-Key: [clave por usuario] |
| Modelo a llamar | claude-sonnet-4-6 via Anthropic API |
| Prompt | System prompt que tú entregas (sección 3.2). Antigravity solo lo pasa al modelo. |
| Deploy | Railway o Render. Entorno de staging primero. |
| Logging | Guardar cada request/response en Supabase para poder revisar y mejorar el prompt. |

## 6.3 Entregable 3 — TRACKFLOW (Fase 3)

Solo arrancar este cuando el backend está validado. La extensión Chrome es una capa de UI sobre una llamada HTTP — si el backend falla, la extensión falla. El orden importa.

 **Punto crítico para Antigravity:** La extensión NO debe procesar el HTML. Su único trabajo es capturarlo y enviarlo. Todo el análisis e interpretación del HTML sucede en el backend (en el prompt). Si Antigravity empieza a añadir lógica de interpretación en la extensión, el sistema se vuelve imposible de mantener.

## 6.4 Lo que no le delegues a Antigravity

| No delegar | Por qué |
|---|---|
| El prompt del sistema | Es el conocimiento propietario de ESES Agency. Antigravity construye el contenedor — tú pones el conocimiento. Si lo delegan, el prompt será genérico. |
| La decisión de qué eventos trackear | Esta es la parte de criterio que distingue a ESES de cualquier herramienta genérica. Debe permanecer en manos del analista. |
| La validación de las implementaciones generadas | Al menos durante las primeras semanas, cada output del modelo debe ser revisado por un analista antes de usarse con un cliente real. |

# 7. Riesgos y decisiones pendientes

## 7.1 Riesgos técnicos

| Riesgo | Cómo mitigarlo |
|---|---|
| El modelo alucina código incorrecto | Los few-shot examples del prompt reducen esto significativamente. Añadir validación en el backend que detecte patrones prohibidos (regex, .includes()) y los marque como warnings. |
| El HTML del cliente es demasiado complejo | El modelo no siempre puede inferir la estrategia correcta. Solución: añadir un campo 'notas del analista' en el payload donde se puede dar contexto adicional. |
| La extensión Chrome rompe en ciertos sites | Content Security Policy en algunos sites bloquea la inyección de scripts. Documentarlo como limitación conocida v1 y abordar en v2. |
| El prompt se queda corto para casos nuevos | Es inevitable. El sistema debe tener un flujo de retroalimentación: cuando una implementación generada es incorrecta, el analista puede marcarla y ese caso se añade al prompt. |

## 7.2 Decisiones pendientes antes de arrancar

| Decisión | Opciones |
|---|---|
| ¿Dónde se aloja el prompt del sistema? | Opción A: hardcodeado en el backend (simple, menos flexible). Opción B: en Supabase, editable sin deploys (recomendado para poder iterarlo rápido). |
| ¿Cómo se comparte con el equipo? | Opción A: extensión Chrome publicada en Chrome Web Store (requiere revisión de Google, ~1 semana). Opción B: instalación manual por desarrollador (.crx) para uso interno mientras se itera. |
| ¿Se factura por uso a clientes? | Si GTMXpert se convierte en un servicio facturado, hay que añadir tracking de uso por cliente desde el principio. Si es solo interno, no hace falta todavía. |
| ¿Qué pasa con los datos de los clientes? | El HTML capturado puede contener información sensible. Definir política de retención de logs: ¿cuánto tiempo se guardan los requests? ¿Están cifrados? |

## 7.3 El MVP mínimo que tiene valor real

 **Definición del MVP:** Un script Python o Node.js local (sin extensión, sin app web) que lee un payload JSON pegado a mano, llama al backend con el prompt del sistema, y

devuelve la implementación en la consola. Con esto, el equipo puede usar GTMXpert en producción desde la semana 4, antes de que exista ni la extensión ni la app.

**Por qué empezar así:** el valor de GTMXpert no está en la UI bonita. Está en que el prompt genera implementaciones correctas. Validar eso primero — con el mínimo de código posible — antes de invertir semanas en la extensión Chrome y la app web.

# 8. Próximos pasos concretos — esta semana

| Tarea | Responsable y cuándo |
|---|---|
| 1. Validar el formato del payload (sección 1.2) con Antigravity | ESES + Antigravity · Esta semana |
| 2. Redactar el prompt v1 (sección 3.2) | ESES · Esta semana (empieza con las reglas R01–R10, luego añade los ejemplos) |
| 3. Testear el prompt v1 manualmente en Claude.ai con 5 casos reales | ESES · Semana 1-2 |
| 4. Antigravity arranca el backend (endpoint POST /generate) | Antigravity · Semana 2 |
| 5. Primera integración: llamar al backend desde Postman con payload real | ESES + Antigravity · Semana 3 |
| 6. MVP en script local: leer JSON → llamar backend → mostrar resultado | ESES o Antigravity · Semana 3-4 |
| 7. Primer cliente real procesado con GTMXpert (aunque sea en modo manual) | ESES · Semana 4 |

 **Una última cosa:** La herramienta más útil que puedes construir no es la más compleja. Es la que tu equipo **usa en cada proyecto**. Empieza pequeño, valida que el prompt funciona bien, y añade capas de UI solo cuando el núcleo esté sólido.